Managing Local and Common Users

Learning Objectives

- Describe the difference between Local and Common Users
- Create common and local users
- Grant privileges commonly and locally
- Describe the difference between common and local roles

Core Concepts: Local vs. Common Users

Local User	Common User
Defined at the PDB level. Not known outside its PDB.	Must be created in the Root container. Known by every existing and future PDB.
Can connect only to the PDB it belongs to.	Can connect to any container (Root or PDBs) if it has privileges.
Cannot be created in the root container.	Username must start with the value of `COMMON_USER_PREFIX` (default **C##**).
Local admin powers do not extend beyond its PDB.	Creating a common user alone gives it no privileges on any PDB.

Recommendation: Keep the default `c##` prefix for common users. It makes distinguishing between common and local users easy and prevents conflicts when moving PDBs.

Creating Users

Create a Common User (from the Root)

```
CREATE USER c##user_admin IDENTIFIED BY "password"
CONTAINER = ALL; -- ALL is optional here, as it's the only valid value for
common users
```

Create a Local User (inside a PDB)

```
-- First, switch to the target PDB
ALTER SESSION SET CONTAINER = pdb1;

CREATE USER app_user IDENTIFIED BY "password";
-- CONTAINER = CURRENT is the default
```

View User Information

```
SELECT username, common, con_id FROM cdb_users WHERE common = 'YES';
```

Granting Privileges

Local Grant (to a local user, inside a PDB)

```
GRANT create session TO app_user; -- CONTAINER = CURRENT is default
```

Local Grant to a Common User (inside a PDB)

```
GRANT create table TO c##user_admin; -- This privilege applies only in the
current PDB
```

⭑ Common Grant (given to a common user, from the Root)

```
-- Grant applies in ALL PDBs associated with the Root
GRANT create any table TO c##user_admin CONTAINER = ALL;
```

⚠ **Important:** A common grant (CONTAINER = ALL) can only be given to a common user by a common user. It applies in all PDBs within the CDB.

Common vs. Local Roles

Nearly the same principles as users apply to roles.

Common Role	Local Role
Created in the Root. Name must start with `c##`.	Created in a PDB. Usable only within its PDB.
Known to all existing and future PDBs.	Cannot be created by local users.
Best Practice: Common roles for common users; local roles for local users. Avoid granting local roles to common users.	

```
-- Create a common role
CREATE ROLE c##admin_role CONTAINER = ALL;

-- Create a local role (inside PDB)
CREATE ROLE app_role;
```

Querying Container Data Objects

Learning Objectives

- Understand querying container data objects
- Set `CONTAINER_DATA` attributes for a common user
- Obtain information about `CONTAINER_DATA` attributes
- Remove `CONTAINER_DATA` attributes for a user

What are Container Data Objects?

These are views/tables (like `V$` views and `CDB_` views) that contain data from multiple containers. They always include the `CON_ID` column to identify the source container.

- **CON_ID = 0**: Information about the entire CDB
- **CON_ID = 1**: Root container
- **CON_ID = 2**: Seed container (PDB$SEED)
- **CON_ID >= 3**: User-created PDBs

```
-- Find all container data objects
SELECT owner, view_name FROM cdb_views WHERE container_data = 'YES';
```

Default Visibility & the CONTAINER_DATA Attribute

By default, a common user (non-Oracle maintained) querying a container data object from the Root sees only the data of the **current container**. This is controlled by the `CONTAINER_DATA` attribute.

Oracle maintained users (SYS, SYSTEM) see all containers by default.

⚙ Setting CONTAINER_DATA

```
-- Allow a user to see data for ALL containers
ALTER USER c##user_admin SET CONTAINER_DATA = ALL;

-- Restrict to specific containers (Root and PDB1)
ALTER USER c##user_admin SET CONTAINER_DATA = (CDB$ROOT, PDB1);
```

```oracle
-- Add a PDB to the list
ALTER USER c##user_admin ADD CONTAINER_DATA = (PDB2);

-- Remove a PDB from the list
ALTER USER c##user_admin REMOVE CONTAINER_DATA = (PDB1);
```

⚠ **SET vs. ADD/REMOVE:** Using `SET` overwrites the entire list. Use `ADD` or `REMOVE` to modify it without losing existing settings.

Object-Level Container Data

You can restrict visibility to specific objects using the `FOR` clause.

```oracle
-- Allow c##user_admin to see only SESSION info for Root and PDB1
ALTER USER c##user_admin SET CONTAINER_DATA = (CDB$ROOT, PDB1) FOR
sys.V_$SESSION;
```

V_$SESSION is the internal object; `V$SESSION` is a synonym.

Viewing CONTAINER_DATA Settings

```oracle
SELECT username, owner, object_name, all_containers, container_name
FROM cdb_container_data;
```

This view shows which users have custom container data settings, for which objects, and which specific containers are included.

Principle of Least Privilege

Learning Objective

- Understand applying the Principle of Least Privileges in Oracle databases

Core Idea: Grant users only the minimum privileges necessary to perform their job. This minimizes security risks.

Oracle's Recommended Tips

1. **Protect Data Dictionary:** Keep `07_DICTIONARY_ACCESSIBILITY` = `FALSE` (default). This prevents users with `ANY TABLE` privileges from modifying dictionary tables.
2. **Restrict UTL Packages:** Do not grant `EXECUTE` on `UTL_SMTP`, `UTL_TCP`, `UTL_HTTP`, `UTL_FILE` to `PUBLIC`. Grant to specific roles instead.
3. **Secure Directory Objects:** Be very cautious when granting access to OS directories via directory objects. Grant only on a business need basis.
4. **Limit Admin Privileges:** Never grant `DBA` role to application accounts. Grant only the specific privileges the app needs.
5. **Remote Authentication:** Keep `REMOTE_OS_AUTHENT` = `FALSE` (default). Change only if absolutely required.
6. **Audit Critical Ops:** Set `AUDIT_SYS_OPERATIONS` = `TRUE` to audit top-level operations.
7. **Avoid Public DB Links:** Discourage the use of public database links. Prefer private links.
8. **Document Everything:** Any grant/revoke workflow should be documented, approved, and every action recorded for evidence.

> **Note:** These are guidelines, not strict procedures, but following them greatly enhances security.

Administrative Accounts and Privileges

🎯 Learning Objectives

- Describe the administrative accounts and privileges
- Describe the difference between SYSDBA and SYSOPER privileges
- Describe the difference between SYS and SYSTEM accounts

🔐 Administrative Accounts

Account	Privilege Keyword	Primary Use
SYS	SYSDBA	Super user. Owns the data dictionary. Can start up/shut down. Password in password file only (12.2+).
SYSTEM	-	DBA role, but **cannot** start up/shut down. Password in data dictionary. Cannot connect if instance is not open.
SYSBACKUP	SYSBACKUP	For backup and recovery operations.
SYSDG	SYSDG	For Data Guard operations.
SYSKM	SYSKM	For TDE keystore management.
SYSRAC	SYSRAC	For RAC (Clusterware) management.

💡 **Key Point:** Except for SYSTEM, administrative accounts require the corresponding keyword (e.g., `AS SYSDBA`, `AS SYSBACKUP`) when connecting. Their passwords are stored in the password file.

⚔️ SYSDBA vs. SYSOPER

SYSDBA	SYSOPER

Full administrative power. Can grant/revoke any admin privilege.	Basic operational tasks (startup, shutdown, backup, recover). **Cannot** view user data. **Cannot** grant/revoke privileges.
Both allow access even when the database is not open. Both require a password file.	

```
-- Granting SYSDBA to a user
GRANT sysdba TO scott;
```

Default Schema for Admin Privileges

When connecting with an admin privilege, your **current schema** changes, which is important for auditing.

Connect As...	Current Schema Becomes...
SYSDBA	SYS
SYSOPER	PUBLIC
SYSBACKUP	SYS
Normal User	The user itself (e.g., SCOTT)

⚠ **Audit Implication:** If user `c##adam` connects AS SYSDBA, all actions are recorded as being done by `SYS`, not `c##adam`.

SYS vs. SYSTEM: The Difference

Feature	SYS	SYSTEM
DBA Role	Yes	Yes

Owns Data Dictionary	✅ Yes	❌ No
Can STARTUP/SHUTDOWN	✅ Yes (via SYSDBA)	❌ No
Password Storage	Password File (only)	Data Dictionary
Login when instance is in MOUNT state	✅ Yes	❌ No

Best Practice: Separate Admin and App Accounts

If a user needs both application privileges and administrative privileges, create **two separate accounts** for them. Do not grant admin privileges to the application account.

> **Example:** User 'Scott' manages application roles via 'scott_app' and performs backups via 'scott_backup' (granted SYSBACKUP). This follows least privilege and improves audit clarity.

Lecture notes compiled from Oracle Database Security training. Always follow documented, approved workflows for grants and privilege changes.